

Training an XGBoost SSH Brute-Force Detector

A step-by-step account of how a binary XGBoost classifier was trained on the cleaned CICIDS2018 dataset to separate normal traffic (Benign) from SSH brute-force attacks — the data used, the feature preparation, the train/test split, the settings, and the results.

Model XGBoost gradient-boosted trees	Features 70 after dropping 8 all-zero	Training rows 874,920 + 218,731 test	Trees built 534 early-stopped from 1000	Test accuracy 100% 0 false positives
---	---	---	---	---

01 — OBJECTIVE & DATA SELECTION

A focused binary problem: normal vs SSH brute force

The model answers one question per network flow: is this **normal traffic** or an **SSH brute-force attack**? Every other attack type in the dataset is ignored, so the two other classes are filtered out before training. Only the Benign and SSH-Bruteforce rows of the cleaned CSV are used.

Class (label)	Available	Used for training
SSH-Bruteforce (1)	94,041	94,041 — all of them
Benign (0)	10,543,008	1,000,000 — random sample

All 10.5M Benign rows do not fit in this machine's memory, so a random 1,000,000-row Benign sample is drawn while the CSV is read in chunks. The full set of 94,041 SSH rows is kept. Final working set: **1,093,651 rows**.

02 — FEATURE PREPARATION

78 inputs → 70 usable features

The cleaned dataset has 79 columns. The **Label** column is the target, leaving 78 feature columns. Eight of those are zero in every row (they carry no information a tree can split on), so they are dropped at training time — leaving **70 features**. All feature values are read as **float32**, the precision XGBoost uses internally, which halves memory versus float64.

The eight dropped all-zero columns: Bwd PSH Flags, Bwd URG Flags, Fwd Byts/b Avg, Fwd Pkts/b Avg, Fwd Blk Rate Avg, Bwd Byts/b Avg, Bwd Pkts/b Avg, Bwd Blk Rate Avg. The dataset itself keeps them; only the model ignores them.

03 — TRAIN / VALIDATION / TEST SPLIT

Stratified, so every part keeps the same class balance

The working set is split 80% / 20% into train and test, keeping the Benign-to-SSH ratio identical in both (a stratified split). Then 10% of the training part is held out as a validation set used only to decide when to stop adding trees. The test set is untouched until the final evaluation.

Part	Share	Rows	Purpose
Train (fit)	72%	787,428	learns the trees
Validation	8%	87,492	early-stopping signal

Test	20%	218, 731	final score only
------	-----	----------	------------------

Class imbalance is handled with `scale_pos_weight = 10.63` (the ratio of Benign to SSH rows in the training set), so each rare SSH row counts about 10.6× as much as a Benign row during training. No rows are duplicated or discarded to balance the classes.

04 — MODEL SETTINGS

XGBoost hyperparameters

<code>objective</code>	<code>binary:logistic</code>	two-class probability output
<code>n_estimators</code>	<code>1000</code>	upper limit on trees (early stopping ends sooner)
<code>learning_rate</code>	<code>0.1</code>	step size per tree
<code>max_depth</code>	<code>6</code>	tree depth — controls complexity
<code>subsample</code>	<code>0.8</code>	fraction of rows per tree (reduces overfitting)
<code>colsample_bytree</code>	<code>0.8</code>	fraction of features per tree
<code>tree_method</code>	<code>hist</code>	fast histogram-based training
<code>scale_pos_weight</code>	<code>10.63</code>	up-weights the rare SSH class
<code>eval_metric</code>	<code>aucpr, logloss</code>	validation metrics (early stopping follows the last)
<code>early_stopping_rounds</code>	<code>30</code>	stop if logloss doesn't improve for 30 trees
<code>random_state</code>	<code>42</code>	reproducible run

05 — TRAINING PROCEDURE

How the run proceeds, in order

- Load & filter** — read the cleaned CSV in 200k-row chunks, keeping every SSH row and a random Benign sample; convert features to float32.
- Split** — stratified 80/20 into train and test, then hold out 10% of train as validation.
- Drop constant columns** — remove the 8 all-zero columns, leaving 70 features.
- Weight the classes** — compute `scale_pos_weight` from the training class counts.
- Fit with early stopping** — add trees one at a time; after each, score logloss on the validation set. Stop when it hasn't improved for 30 trees and keep the best iteration.
- Evaluate once** — predict on the held-out test set and compute the metrics below.
- Save** — write the model, the feature list, an evaluation report, and the plots.

Why the metric order matters. Early stopping watches the *last* metric in the list. Precision-recall AUC (`aucpr`) reaches 1.0 on the very first tree here, so if it were last, training would stop at ~2 trees. Putting logloss last lets training continue and refine to 534 trees, which produces well-calibrated probabilities.

06 — RESULTS

Held-out test set of 218,731 flows

Confusion matrix		
	pred Benign	pred SSH
actual Benign	199,923	0
actual SSH	0	18,808

Metrics	
Accuracy	1.0000
Precision (SSH)	1.0000
Recall (SSH)	1.0000
F1 / ROC-AUC / PR-AUC	1.0000
False positive rate	0.0000

Across 218,731 test flows the model made **zero** mistakes: no normal flow flagged as an attack, and no attack missed.

07 — WHAT THE MODEL LEARNED

Top features by gain (contribution to splits)

Feature	Gain
Fwd Act Data Pkts	74,601.9
Fwd Header Len	45,944.1
Dst Port	948.9
Fwd Seg Size Min	460.4
ACK Flag Cnt	361.7
URG Flag Cnt	293.4
Bwd Pkts/s	287.6
Fwd Pkts/s	56.8

The model leans on packet-count and header-length features: SSH brute-force flows are many short, near-identical login attempts, which look very different from normal SSH sessions in these counts.

Read the perfect score with care. CICIDS2018's SSH brute-force flows are extremely uniform, so this dataset is easy to separate and a 100% test score is expected. It does **not** promise the same accuracy on live traffic from a different network — validating that is the purpose of a real-time lab test.

08 — OUTPUTS & USING THE MODEL

Saved files and how to load them

Training writes to **xg_bruteforce/**: `ssh_bruteforce_xgb.json` (the model), `model_features.json` (the 70 feature names, in order, plus the 0.5 threshold), `evaluation_report.txt`, and two plots.

```
import json, pandas as pd, xgboost as xgb

model = xgb.XGBClassifier()
model.load_model("xg_bruteforce/ssh_bruteforce_xgb.json")
feats = json.load(open("xg_bruteforce/model_features.json"))["features"]

df = pd.read_csv("new_flows.csv")          # same CICFlowMeter columns
pred = model.predict(df[feats])           # 0 = Benign, 1 = SSH brute force
proba = model.predict_proba(df[feats])[:, 1] # attack probability
```

Libraries: Python 3.13 · xgboost 3.4.1 · scikit-learn 1.9 · pandas 3.0 · numpy 2.5 · matplotlib. On this machine, disable the hanging WMI query before importing pandas / xgboost.

Figures generated from evaluation_report.txt and model_features.json. Report compiled 2026-09-28.